

Sistemas Operativos

Departamento de Computación - FCEyN - UBA

Primer cuatrimestre de 2025

Primer parcial - 1er cuatrimestre de 2025

Nombre y apellido: Tomás Rodríguez
Nº orden: 106 L.U.: 431/23 Cant. hojas: 6

1	2	3	4	Nota
18	15	25	25	83

(A)

- Cada ejercicio debe realizarse en hojas separadas y numeradas. Debe identificarse cada hoja con nombre, apellido, LU y número de orden. Hojas sin identificar no serán corregidas. Numere las hojas entregadas. Complete en la primera hoja la cantidad total de hojas entregadas. Entregue esta hoja junto al examen, la misma no se incluye en la cantidad total de hojas entregadas. La devolución de los exámenes corregidos es personal. Los pedidos de revisión se realizarán por escrito, antes de retirar el examen corregido del aula.
- Los parciales tienen dos notas: I (Insuficiente): 0 a 64 pts y A (Aprobado): 65 a 100 pts.

Ejercicio 1. (25 puntos)

Un proceso será el encargado de completar los valores de un arreglo de enteros de m posiciones con valores específicos. Para ello va a crear n procesos hijos, quienes cuentan con la función `int generarValor(int indice)`, que toma como argumento el índice del arreglo que queremos completar, y devuelve un número entero. Esta función sólo puede ser ejecutada en el contexto de procesos hijos. Por último, cuando el padre tenga el arreglo de m posiciones completo y sus hijos hayan terminado, deberá informar su valor utilizando la función `printArray(int[] array)`. Se pide:

- Escribir el pseudocódigo de un programa que cumpla con lo solicitado, utilizando pipes como medio de comunicación entre los procesos, asumiendo que m es múltiplo de n y que los hijos trabajarán de forma equitativa.
- Supongamos que `generarValor()` puede demorar un tiempo variable en cada caso, por lo que algunos procesos hijos pueden terminar su trabajo antes que otros, y queremos que no haya procesos hijos ociosos en caso de que aún queden ítems del arreglo sin procesar. ¿La solución provista en el inciso anterior funciona? En caso afirmativo, justificar. En caso negativo, explicar por qué y proponer una idea de solución.

Ejercicio 2. (25 puntos)

Tenemos un sistema donde existen dos tipos de procesos:

- **Proceso Tipo 1:** Recibe una solicitud de un usuario y agrega un trabajo a una cola de tareas. Esta cola tiene un tamaño máximo N , ya que no se pueden tener demasiadas tareas esperando en la memoria.
- **Proceso Tipo 2:** Toma trabajos de la cola (usando `pop()`) y realiza un trabajo de procesamiento.

Si la cola se llena, no se le deben agregar nuevos trabajos. La cola de trabajos no es atómica. La interacción de cada proceso con la cola se implementó hace poco y su comportamiento se puede representar con el siguiente pseudocódigo:

```
// Variables compartidas:  
mutex = sem_init(1)  
jobs = sem_init(0)  
buffer cola(N) // buffer de tamaño N
```

```
// Proceso Tipo 1  
trabajo = esperarPromptUsuario();  
mutex.wait();  
cola.add(trabajo);  
mutex.signal();  
jobs.signal();
```

```
// Proceso Tipo 2  
mutex.wait();  
jobs.wait();  
trabajo = cola.pop();  
mutex.signal();  
enviarResultado(trabajo);
```

Sin embargo, se reportó que esta parte del código no está funcionando correctamente y que no cumple exactamente con lo pedido. Nos encargaron la tarea de identificar qué está ocurriendo. Se pide:

- Encontrar qué problema/s tiene este código con respecto a la sincronización de los procesos. Justificar detalladamente la/s causa/s y corregirlo, cumpliendo con la consigna del problema.
- Ahora se desea que k procesos Tipo 2 (siendo k mucho menor que N) trabajen concurrentemente, pero sólo puedan enviar un resultado cuando k procesos hayan extraído una tarea de la cola. Es decir, ningún proceso puede enviar el resultado inmediatamente después de retirar el trabajo de la cola, debe esperar a los otros procesos. Extender la solución del inciso anterior con esta restricción, asegurando que no haya problemas de concurrencia. Justificar la solución.

Ejercicio 3.(25 puntos)

Se quiere avanzar en el diseño de un sistema de control para un dron autónomo de rescate. Este sistema debe gestionar múltiples tareas simultáneamente para garantizar un funcionamiento eficiente del dron. El dron debe ser capaz de ejecutar tareas con periodicidad fija y *deadlines* estrictos (se garantiza que siempre se pueden cumplir), tales como el escaneo de obstáculos y la estabilización de vuelo.

Además, el sistema debe poder procesar *logs* en segundo plano sin afectar el rendimiento general del sistema. También debe ser capaz de gestionar tareas disparadas remotamente por el operador, las cuales pueden tener diferentes niveles de prioridad (*urgente, media, baja*), permitiendo al operador priorizar ciertas tareas según la urgencia.

- Proponer un esquema de *scheduling* para este sistema que tenga en cuenta todas las restricciones y requerimientos. Justificar detalladamente cada decisión.
- La siguiente tabla representa el comportamiento de las tareas periódicas con *deadlines* específicos. Dibujar el diagrama de Gantt resultante de aplicar el esquema descrito en el inciso anterior, desde el momento 0 hasta los 25 ms.

Tarea	Descripción	T. ejecución	Período	Deadline relativo	T. inicial
T1	Sensado de obstáculos	1 ms	5 ms	5 ms	0
T2	Estabilización del vuelo	2 ms	10 ms	7 ms	0
T3	Actualización de posición GPS	1 ms	20 ms	20 ms	0
T4	Comunicación con central de mando	2 ms	25 ms	20 ms	5

Ejercicio 4.(25 puntos)

Considerar el siguiente programa en C:

```
int X[N];
int step = M; // M es una constante predefinida
// ...
for (int i = 0; i < N; i += step){
    if (esPar(i)){
        X[i] = X[i] + 1;
    }else{
        X[N-i] = X[i] + 1;
    }
}
```

Sabemos que cada página de memoria puede contener exactamente P enteros, la memoria física tiene F marcos de página y cada lectura o escritura a $X[i]$ implica un acceso a memoria.

- Dado $N = 16$, $M = 3$, $P = 4$, listar la secuencia de números de página accedidos y justificar cómo se derivan esos números de página a partir de los valores de i . Considerar que el código y los valores de M e i se encuentran en una página cargada que no puede desalojarse, y que $X[0]$ se encuentra al inicio de la página 1.
- Usando $F = 3$ marcos de página y la secuencia de pedidos calculada en el inciso anterior, simular los algoritmos de reemplazo FIFO y Second Chance. Para cada uno, indicar la cantidad total de fallos de página y el contenido de los marcos tras cada acceso.
- ¿Cómo cambia la cantidad de fallos de página si $M = 2$ (manteniendo $N = 16$, $P = 4$, $F = 3$)?

1) void padre() {

int pipes[N][2]; // Crea lugar para los pipes con cada hijo

int array[M]; // Crea el arreglo para guardar los resultados

for (int i=0; i<N; i++) {

pipe(pipes[i]); // Crea el i-esimo pipe

if (fork() == 0) {

cerrarPipes(pipes, i); // Cierro todos los pipes menos el propio

proceso_hijo(i, pipes[i]);

}

}

// A partir de aca es solo código del proceso padre

// Para ser más declarativo uso WRITE=1, READ=0

// Ahora que mi idea es que el i-esimo hijo defina los

valores $j: 0 \leq j \leq M$ para $j \% N = i$, el valor que le pasamos al hijo.

int sol_tmpe;

for (int i=0; i<M; i++) {

read(pipes[i % N][READ], &sol_tmpe, sizeof(int));

// Leo el i-esimo valor de la solución.

array[i] = sol_tmpe; // Lo guardo en el arreglo

}

// Cierro los pipes y espero a los hijos para liberar ^{sus} recursos

// del ~~proceso~~ sistema

for (int i=0; i<N; i++) {

close(pipes[i][READ]); close(pipes[i][WRITE]);

wait(0);

}

// Printeo el resultado pedido

Print Array(array);

exit(0)

}

Corrección: Hermano G.

~~Ahora escribo las funciones faltantes:
 void cerrarPipes(~~array~~ int pipes[2][2], int indice){
 for (int i=0; i<indice; i++){
 if (i != indice){
 close(pipes[i][READ]);
 close(pipes[i][WRITE]);
 }
 }
 }~~

~~Simplemente recorre todas las pipes, salvo el que te
 indican que es solo.~~

~~void Proceso-hijo{~~

~~void cerrar_pipes(int pipes[2][2], int indice){
 for (int i=0; i<indice; i++){
 close(pipes[i][READ]);
 close(pipes[i][WRITE]);
 }
 }~~

Cierra todos los pipes anteriores al propio. Esto es
 porque al llamar a esta función, los pipes posteriores
 al propio no habrán sido abiertos y tiene barra en el resto
 del arreglo.

void Proceso-Hijo (int ind, int pipes[2]) {
 close (pipes[READ]); // porque ~~el~~ no va a leer nada
 int valor;
 for (int j=ind; j<M; j+=N) {
 valor = generarValor(j); // crea el valor para array[j]
 write(pipes[WRITE], &valor, sizeof(int));
 // crea el valor y lo envia al padre.
 }
 close (pipes[WRITE]);
 exit(0);
 }

Notar que como sumamos N al índice, se guina la lógica que propuse al inicio de que cada proceso calcule los valores de los índices $1 \leq j \leq M$ y $j \% M = \text{índice}$, porque $\text{índice} + N \% M = \text{índice}$.

Esto distribuye la carga de forma equitativa entre procesos, pues cada uno genera $\frac{M}{N}$ valores.

También es importante el exit(0) al final de la función, caso contrario seguiría con el código del padre.

b) La solución del inciso a) funciona, porque el proceso padre lee los resultados de los hijos al estilo Round-Robin.

Funcionaría de igual manera si no lo hiciera porque no depende de cuando escribe en los pipes. Pero de esta manera, o menos, que la diferencia de tiempo no excesiva, como el padre lee los pipes "en ronda" (es decir que lee de un pipe y lee los otros $M-1$ pipes antes de volver a leer el mismo) probablemente no afecta a la ejecución del padre que ahora generar valores tome más tiempo.

x6) no, la idea era que los hijos procesen "todos los índices que puedan" y no un rango prebijado.

Q2) a) Este código puede generar Deadlocks.

Digamos que estamos en el inicio de ejecución del programa donde $\text{mutex} = 1$, $\text{Jobs} = 0$.

Ahora un proceso de tipo 2 comienza a ejecutarse antes que ~~los~~ los de tipo 1 lleguen al mutex.wait() .

Va a pasar este wait porque $\text{mutex} = 1$ y lo setea en 0. Pero como $\text{Jobs} = 0$, se queda esperando en Jobs.wait() .

Pero para que se ejecute el proceso de tipo 1 y envíe el Jobs.signal , necesita que mutex sea 1. Pero nunca se puede setear en 1 porque para eso el proceso de tipo 2 se ~~se~~ ~~espera~~ debe seguir ejecutando.

Así que llegamos a una situación donde el proceso tipo 1 necesita que el de tipo 2 se siga ejecutando y viceversa. En otras palabras deadlock.

Para resolverlo solo es necesario modificar el código de los procesos de tipo 2:

```
Jobs.wait();
```

```
mutex.wait();
```

```
trabajo = cola.Pop();
```

```
mutex.signal();
```

```
enviar Resultado(trabajo);
```

De esta manera, los procesos de tipo 2 solo pueden llegar al mutex si $\text{Jobs} > 0$. De esta manera, al salir de mutex.wait() no tienen que esperar nada más, tienen asegurado que hay trabajo y van a llegar al mutex.signal() .

Sigo del otro lado.

FALTA IDENTIFICAR QUE EL TAMAÑO DEL BUFFER DEBE SER MANEJADO, NO PUEDE SUPERAR DE N TAREAS

b) Para esto creo un nuevo semáforo

~~resultado = sem-init(0)~~ enviar = sem-init(0)

y una variable global

~~int resultado = 0~~ int resultados = 0

Solo modifico el código de los procesos de tipo 2.

```
Jobs.wait();
```

```
mutex.wait();
```

```
trabajo = cola.pop();
```

```
resultados++;
```

```
if (resultados == == K) {
```

```
    for (int i = 0; i < K; i++) {
```

```
        enviar.signal();
```

```
    }
```

```
} RESULTADOS = 0
```

```
mutex.signal();
```

```
enviar.wait();
```

```
enviarResultado(trabajo);
```

La idea es que vamos guardando la cantidad de trabajos extraídos de la cola en resultados.

~~Se le da un proceso que extrae los~~

Luego se quedan esperando al semáforo de enviar.

Cuando un proceso lee que ya hay K trabajos extraídos, envía K señales al semáforo enviar, uno por cada proceso de tipo 2.

Es importante hacerlo dentro del mutex, porque las sumas y comparaciones en resultado podrían generar race conditions. Caso contrario, por ejemplo si 2 procesos quieren sumar 1 a resultado a la vez o incluso si resultado fuera atómico, podría pasar que 2 procesos le sumen 1 y luego ambos lo comparan a K, enviando ~~señales~~ 2K señales.

Si nos interesa que se ejecuten varias veces los procesos,

Tomás

Lvi 431/23

Hoja 1

Rodriguez Rodin

Nº ordeni log

podemos agregar ~~return~~
resultados = 0;

dentro del if (resultados == K)

Así al reiniciar la ejecución, resultados vuelve a ser 0.

3) Para este sistema, propondría como algoritmo de scheduling una multi-level que sea preemptive, donde menor número de cola significa mayor prioridad *¿Por qué?*

- Cola 0: Procesos con deadlines, usando algoritmo EDF. como estas tareas son cruciales (si el dran no esquivan, se rompe y no puede hacer nada) deben ser atendidas antes de sus deadlines y por eso tienen más prioridad que el resto.

- Colas 1, 2 y 3: Tareas disparados por el operador por orden de prioridad (Cola 1: urgente, Cola 2: media, Cola 3: baja). Sobre estas colas usaría el algoritmo FCFS. Esto es porque no nos interesa un algoritmo justo, sino que responda lo antes posible a las tareas indicadas y probablemente, al ser instrucciones, que las ejecute en orden (salvo por urgencia).

- Cola 4: Logs en segundo plano. Algoritmo FCFS. Estas tienen la menor prioridad porque sobre todo nos interesa que no afecte el rendimiento del sistema.

uso FCFS porque no nos interesa que sea justo, solo que guarde los logs ~~en orden~~ probablemente en orden

¿Puede generar starvation? Sí, si llegan muchos procesos RT o de gran urgencia.

No podemos hacer nada respecto a los RT, pues tienen un deadline estricto. Además, los procesos RT suelen tener más pagos E/S, así que es difícil llegar a este punto.

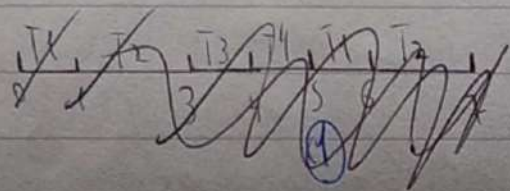
Sobre los procesos disparados por un operador, podemos agregar algo de las colas 1 a 3, donde si pasan mucho tiempo sin ejecutarse los subimos una cola.

no lo extiendo hasta la cola 9 porque nos interesa que las tareas sobre las que no afectan el rendimiento del sistema.

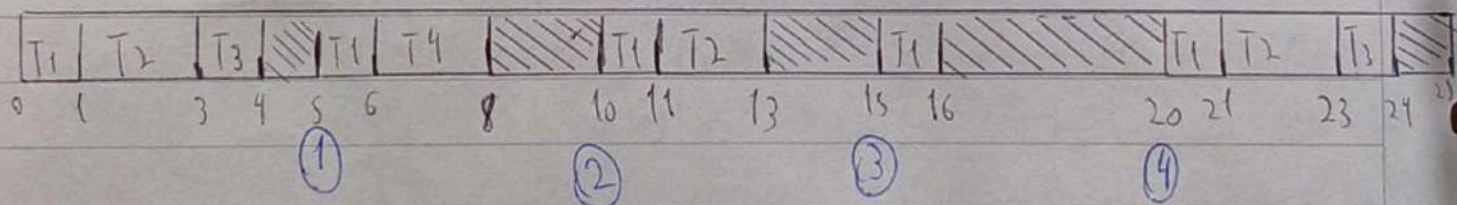
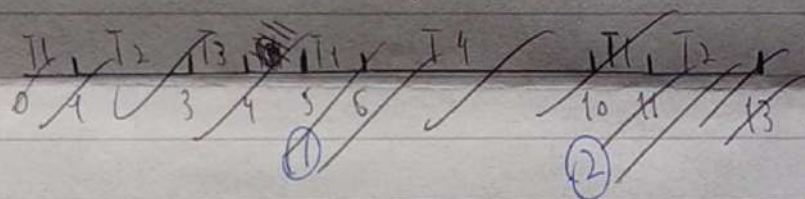
b) Primero notemos los momentos importantes para hacerlo

- ① 5ms: vuelve a la cola T_1 , con un deadline en 10ms. T_4 en 20ms
- ② 10ms: vuelven a la cola T_1 y T_2 , con deadlines en 15ms y 17ms respectivamente
- ③ 18ms: vuelve T_1 con un deadline en 20ms
- ④ 20ms: vuelven T_1 con un deadline en 25ms, T_2 con un deadline en 27ms y T_3 con un deadline en 40ms

Todos estos procesos están en la cola 0, con el algoritmo EDF



Perdón por tachar tanto,
me mezclé los números
No pasa nada. ✓



Los periodos en rayas son donde no había procesos real time en la cola. No significa necesariamente que no hacía nada el CPU, podría haber estado trabajando con procesos de menor prioridad.

Aclaración: olvide ponerlo antes, pero en el instante o momento en la cola:

- T_1 con deadline en 5ms
- T_2 con deadline en 7ms
- T_3 con deadline en 20ms

Tomás
Rodríguez Madín

Lvi 931/23
nº ordeni 106

Hoja 6

4) a) Como $p=4$ y $n=16$, sabemos que X está separado en 4 páginas:

1: {0-3} 2: {4-7} 3: {8-11} 4: {12-15}

índice
de página de X

Como $m=3$, los valores de i en el for van a ser
 $i = 0, 3, 6, 9, 12, 15$

veremos qué valores de X se accede por cada i :

i	índice de X	páginas	2 por cada i
0	0 y 0	1 y 1	1 para leer $X[i]$
3	3 y 13	1 y 4	y otro para modificar
6	6 y 6	2 y 2	$X[i]$ o $X[n-i]$ según
9	9 y 7	3 y 2	corresponda
12	12 y 12	4 y 4	
15	15 y 1	4 y 1	

Luego los accesos de página son:

1 1 1 4 2 2 3 2 4 4 4 1

b) Asumo que trabajamos con 3 marcos de página solo para los datos, sin tener en cuenta la página para código

	FIFO		Second chance	
Pedidos	Frames	Códs	Frames	Códs
1	1 _ _	1	1 _ _	1
1	1 _ _	1	1 _ _	1
1	1 _ _	1	1 _ _	1
4	1 4 _	1 4	1 4 _	1 4
2	1 4 2	1 4 2	1 4 2	1 4 2
2	1 4 2	1 4 2	1 4 2	1 4 2
3	3 4 2	4 2 3	1 3 2	1 4 2
				1 4 2
				1 2 3
2	3 4 2	4 2 3	1 3 2	1 2 3

pedidos	FIFO		Second chance	
	frames	cola	frames	cola
4	3 4 2 •	4 2 3	4 3 2	1 2 3 ↑ 2 3 4
4	3 4 2 •	4 2 3	4 3 2 •	2 3 4
4	3 4 2 •	4 2 3	4 3 2 •	2 3 4
1	3 1 2	2 3 1	4 1 2	2 3 4 ↑ 2 3 4 ↑ 2 4 1

FIFO tuvo 5 page faults y second chance tuvo 6 page faults.

Aclaración: uso • para marcar hits y marco las páginas en la cola de 2201 para indicar que tienen el bit r prendido.

c) Si $m=2$, los valores de i pasarían a ser $i=0, 2, 4, 6, 8, 10, 12, 14$.

por cada i hay 2 accesos a páginas: uno para leer $X[i]$ y otro para modificar $X[i]$.

Los accesos ~~para~~ de $X[i]$ para cada i son

$i=0, 2$: página 1

$i=8, 10$: página 3

$i=4, 6$: página 2

$i=12, 14$: página 4

Luego los accesos son:

4 veces página 1, 4 veces página 2, 4 veces página 3 y 4 veces página 4.

Es fácil ver que para cualquier algoritmo visto en la materia, solo genera un page fault el primer pedido a cada página y el resto son hits.

por lo tanto hay 4 page faults.